

Programmazione dei calcolatori con laboratorio

7 settembre 2026

Istruzioni per la consegna

- Creare due file, uno per la soluzione della parte in C e l'altro per la soluzione della parte in Python;
- Per i nomi dei file usare il formato "cognome_nome.py" and "cognome_nome.c";
- Creare una archivio chiamato "cognome_nome.zip" contenente i due file;
- Inviare l'archivio per email rispondendo (**reply**) all'email ricevuta senza modificare il soggetto.

NB. Verranno sottratti punti in proporzione ai minuti di ritardo dalla scadenza.

Per ciascuna funzione implementata, descrivere in un commento la complessità temporale e spaziale, motivando in modo chiaro e convincente la risposta.

1) Python

Si scriva una funzione, denominata `find_max_base`, che riceve in ingresso una lista composta da `n` elementi (ciascuno dei quali è una lista o una tupla, potenzialmente annidata) e restituisce l'elemento che contiene il maggior numero complessivo di *elementi scalari*, considerando tutti i livelli di annidamento.

Per *elemento scalare* si intende qualsiasi valore base (es. interi, float, stringhe) che non sia a sua volta una lista o una tupla.

Esempio

```
dati = [  
    [1, 2, 3],           # 3 elementi scalari  
    [1, [2, [3, 4]]],    # 4 elementi scalari  
    [3, [4, 3]]          # 3 elementi scalari  
]
```

```
risultato = find_max_base(dati)  
print(risultato) # Output: [1, [2, [3, 4]]]
```

2) C

Scrivere una funzione in C che prenda in ingresso un numero intero positivo `d` e restituisca una stringa allocata dinamicamente (`char*`) della dimensione esatta necessaria a contenere la sua rappresentazione formattata con i separatori delle migliaia.

La funzione deve rispettare il seguente prototipo

```
char *prepara_buffer_migliaia(int d);
```

Dove `d` è l'intero da cui ottenere la stringa. La funzione deve ritornare il puntatore alla stringa creata oppure `NULL` in caso di errore.

Esempio Se `d = 12345678`, la funzione `prepara_buffer_migliaia(int d)` deve restituire un array di `11 char`.